

GUÍA DE ESTUDIO INTERNA · PARA PREPARAR LA EXPOSICIÓN

Generador Académico **RAG**

Todo lo que hay que entender del trabajo —la propuesta, los conceptos, cómo se construyó, las tecnologías y los resultados— en una lectura corta y clara, antes de la defensa oral.

MATERIA Ciencia de Datos · UTN FRLP · 2026

GRUPO **Grupo 14** · Martin · Gentil · Natalichio · Cuenca · Ocampo · Aubert

CÓMO USAR Leéla entera una vez; después repasá tu sección (§9) y las preguntas del jurado (§10).

01 La propuesta, en una frase

Un sistema de **IA generativa** que, combinando una base de datos **relacional** (nuestras notas y correlatividades reales) con una base **vectorial** (la documentación de la carrera), **genera documentos académicos personalizados** —planes de cursada, informes, cartas— que el modelo **no podría escribir sin ese conocimiento privado**.

La idea central: el modelo de lenguaje no conoce **tus** notas ni el régimen de correlatividades de la UTN. Son datos privados y de nicho. Le damos acceso a ellos con la técnica **RAG** y entonces puede generar respuestas concretas y ancladas en datos reales. El RAG no es un adorno del trabajo: es su corazón.

02 El problema y por qué no es trivial

Usamos **dos bases de datos**, cada una para lo que sirve. Lo **tabular** (notas, años, correlatividades) vive en una base **relacional**, donde un `SELECT` hace búsquedas exactas. Eso está bien y lo aprovechamos tal cual: es la mitad correcta del problema. Pero el producto no es esa consulta.

El producto es lo que **generamos encima** de esos datos: texto nuevo, redactado, con razonamiento y estilo. Un SQL te dice qué materias **podés** cursar; nuestro sistema redacta qué te **conviene** y por qué. Por eso tampoco es un chatbot de preguntas ni NotebookLM: la gracia del TP es que el pipeline RAG lo **construimos nosotros** y lo mostramos funcionando.

CÓMO RESPONDE A LA CÁTEDRA

La cátedra pidió dos cosas: (1) que el foco sea lo **generativo** —generar lo que el modelo no puede solo— y (2) que combinemos **base relacional + vectorial**, sin meter lo tabular en la vectorial. El trabajo hace exactamente las dos. La arquitectura híbrida es la respuesta directa a esa indicación.

03 Conceptos y terminología (glosario mínimo)

Término	Qué es, en una línea
RAG	Retrieval-Augmented Generation . En vez de confiar en la memoria del modelo, recuperamos el contexto justo (de nuestras bases) y se lo damos para que genere anclado en él.
Embedding	Un texto convertido en un vector de números que captura su significado . Textos parecidos → vectores cercanos. Permite buscar por sentido, no por palabra exacta.
Base vectorial (Chroma)	Almacena los embeddings de la prosa del plan. Busca por similitud semántica (lo que un <code>SELECT</code> no puede). Modelo de embeddings: MiniLM, local y offline.
Base relacional (SQLite)	Guarda lo tabular (alumno, materia, nota, año, correlativas). Se consulta con SQL exacto : lookups y joins, sin ambigüedad.
Persistencia híbrida	La decisión de arquitectura clave: cada dato en su base (tabular→relacional, texto→vectorial). No mezclar.
Retrieval híbrido	Para responder, recuperamos de las dos bases a la vez y armamos un solo contexto: SQL ancla las notas reales, los embeddings traen la documentación del plan.
Grounding	Cuánto de la respuesta está anclada en el contexto recuperado (y no inventada). Lo medimos léxico (palabras en común) y semántico (similitud de significado).
Precisión factual	De los pares (materia, nota) que el modelo afirma, qué fracción coincide con el registro real . Mide acierto, no parecido.
Faithfulness	Un LLM actúa de juez y puntúa 0–1 cuánto se apoya la respuesta solo en el contexto. (LLM-as-judge .)
Hit-rate / MRR	Calidad del retriever solo: si la ficha correcta aparece entre los primeros resultados (hit-rate@k) y en qué posición (MRR = 1/rango promedio).
t-SNE	Técnica para proyectar vectores de muchas dimensiones a 2D y verlos. La usamos para mostrar que materias de la misma área quedan juntas.
KMeans / PCA	Clustering (agrupar) de perfiles de los integrantes por fortalezas + proyección a 2D para visualizar.
Tono	Mismo dato, distinto registro de redacción (técnico / motivacional / honesto). Es lo que hace al sistema generativo y no un reporte fijo.
Ollama	El motor que corre el LLM local . Lo usamos con el modelo <code>qwen2.5:14b</code> en GPU (o <code>phi4-mini</code> en CPU).

04 Cómo funciona — la arquitectura



Los PDF se parsean a texto y se reparten: lo **tabular** va a SQLite, la **prosa** se vectoriza con MiniLM y va a Chroma. Ante un pedido, el retrieval **combina ambas** en un solo contexto y el LLM genera anclado en él. Si Ollama está apagado, la parte RAG funciona igual y devuelve el contexto recuperado (modo solo-retrieval).

Por qué híbrido y no todo en la vectorial

Meter las notas en la base vectorial sería usar **búsqueda aproximada** para algo que es un **lookup exacto**: un riesgo de inventar notas. La relacional garantiza cero invención de datos; la vectorial aporta el lenguaje del plan. Separarlas es deliberado.

05 Cómo se construyó y con qué

El pipeline se armó en **módulos independientes**, cada uno con sus propios chequeos (self-checks). El flujo sigue el pipeline de Ciencia de Datos completo: ingesta → preprocesamiento → persistencia → análisis/visualización → generación → evaluación.

Módulo	Qué hace
extract.py	PDF → texto (pdfplumber).
documents.py	Texto → documentos + correlatividades. <code>normalizar()</code> = clave de join entre estado y plan.
db.py	Datos tabulares → SQLite. Consultas SQL (aprobadas, correlativas, registro real).
ingest.py	Prosa + fichas de materia → Chroma. <code>buscar()</code> semántico.
rag.py	Retrieval híbrido (SQL + vector) → LLM vía Ollama. Fallback si está apagado.
generar.py	Los 6 generadores de artefactos con control de tono.
analisis.py / figuras.py	EDA, perfil por área, radar, clustering (KMeans/PCA), t-SNE.
evaluar.py	Grounding léxico+semántico, precisión factual, faithfulness, demo con/sin RAG.

pdfplumber sqlite3 chromadb all-MiniLM-L6-v2 pandas scikit-learn matplotlib

Ollama · qwen2.5:14b FastAPI (web app)

ARQUITECTURA DE EJECUCIÓN

Todo el pipeline corre **local y offline**. Lo único que puede cambiar de lugar es la generación del LLM: `rag.py` habla con Ollama por HTTP, así que el modelo puede correr en la PC (CPU) o en una **GPU T4 gratuita de Colab** por un túnel. Los datos nunca pasan por un proveedor externo: la privacidad se mantiene.

06 Los 6 artefactos generados

Artefacto	Qué genera
Plan de cursada	Qué materias cursar el próximo cuatrimestre, priorizadas y justificadas (combina correlativas + fortalezas + contenidos).
Informe de trayectoria	Relato en prosa de la evolución académica: fortalezas, debilidades, consejo final.
Carta de pasantía	Carta de presentación en primera persona, conectando fortalezas reales con el puesto.
Recomendar orientación	Hacia qué especialización conviene perfilarse, según el promedio por área real.
Diagnóstico grupal	Reparte roles de un proyecto entre los 6 según sus fortalezas (usa el clustering).
Simulación what-if	Proyecta el promedio si se aprueban ciertas materias con ciertas notas.

Todos comparten una regla de prompting: el modelo usa **solo** el contexto recuperado, tiene prohibido inventar, pero debe **interpretar y seleccionar** (no recitar todas las notas). Sobre eso se aplica el **tono**.

07 Evaluación y resultados

La pregunta que medimos: **¿cuánto del contexto usa el modelo, y cuánto inventa?** La **demo estrella** corre el mismo pedido **con RAG** (contexto real) vs **sin RAG** (sin contexto). Promediando 4 artefactos con qwen2.5:14b:

1.0	0.75 / 0.25	0.69 / 0.22	0.60 / 0.53
precisión factual con RAG (cero notas inventadas)	faithfulness (juez LLM) con / sin RAG	grounding léxico con / sin RAG	grounding semántico con / sin RAG

En las 4 métricas, **con RAG > sin RAG**: evidencia medible de que el conocimiento privado es el corazón del producto. Honestidad metodológica: la generación es estocástica, así que el valor absoluto de una corrida varía; lo robusto es la **dirección** (siempre con>sin) y la precisión factual.

Calidad del retriever (recuperación sola, sin LLM)

Aparte medimos solo la recuperación con **hit-rate** y **MRR** sobre 12 consultas-tema: **hit-rate@1 = 0.92 · MRR = 0.92**. Estos números mejoraron limpiando el texto que se vectoriza (sacar el boilerplate que las 36 fichas repetían y dejar solo nombre + contenidos mínimos): mejor

preprocesamiento, no truco. Validamos además que un embedder multilingüe (e5-large) lleva la recuperación a **1.00 perfecto**; quedó como opción por su peso (2.2 GB) frente al MiniLM liviano del default.

08 Entregables del trabajo

- **Notebook de Colab** — el end-to-end de todo el pipeline, ejecutable de punta a punta (pide el `.zip` y descomprime solo).
 - **Web app** — consola en vivo de la demo con/sin RAG; terceriza la generación a la GPU de Colab.
 - **Informe técnico** — el documento formal con los 5 apartados.
 - **Presentación** (slides) + **este guión** seccionado por integrante.
 - **.zip de entrega** — todo lo necesario para correr el notebook en Colab.
-

09 Mapa de la exposición — quién dice qué

Turno	Tema	Slides
1 · Tomás	Apertura + el problema (generar, no consultar)	1–2
2 · Mora	Datos, corpus mixto y EDA (por qué dos bases)	3
3 · Santiago	Pipeline RAG + espacio semántico (embeddings, t-SNE)	4–5
4 · Juan Bautista	Generación, prompting y los 3 tonos	6
5 · Simón	Demo con/sin RAG + evaluación (métricas)	7–8
6 · Lautaro	Diagnóstico grupal + conclusiones	9–10

Reparto de preguntas: pipeline técnico → Santiago · datos → Mora · evaluación → Simón. El guión completo (`guion/`) trae el speech y las "claves si te preguntan" de cada uno.

10 Preguntas probables del jurado

Si preguntan...	Respuesta corta
"¿Esto no lo hace un SQL?"	El SQL resuelve el lookup exacto, y por eso lo tabular va en la relacional. Pero el producto es el texto generado encima (plan, carta), que un SQL no redacta.
"¿Por qué dos bases de datos?"	Cada dato en la que le corresponde: tabular → relacional (exacto), prosa → vectorial (semántico). El agente combina ambas. Es la indicación de la cátedra.
"¿Qué es el grounding?"	Cuánto de la salida está anclada en el contexto recuperado. Lo medimos léxico y semántico, más precisión factual y faithfulness.
"¿Cómo evitan que invente notas?"	El system prompt obliga a usar solo el contexto; la relacional ancla las notas exactas. Medido: precisión factual 1.0 con RAG.
"¿Y la lematización / limpieza de texto?"	Con embeddings sub-palabra (MiniLM) no hace falta lematizar; la limpieza relevante es el chunking de la prosa y la normalización de nombres para el join.
"¿Corre sin internet / es privado?"	Sí, todo local y offline. Solo la generación puede ir a la GPU de Colab; los datos no pasan por terceros.
"¿Los datos son reales?"	Sí: notas e historiales reales de los 6 integrantes (parseados de PDFs del campus). Los perfiles nunca son inventados.